

Computação Autônoma

Redes de Computadores I

Marco Antônio Cottorello Henry Murilo de Souza Neves

Universidade Estadual de Londrina

2026

Sumário

Histórico e Desenvolvimento

Sistemas Multitier e Graus de Autonomia

Aplicações

Estado da Arte

Conclusão

Sumário

Histórico e Desenvolvimento

Aplicações

Estado da Arte

Conclusão

Motivação: A Crise da Complexidade

- ▶ Sistemas modernos: distribuídos, heterogêneos, em nuvem — componentes de múltiplos fornecedores interagindo em tempo real
- ▶ **TCO**: até 80% do custo de TI em manutenção, não em hardware
- ▶ **Erros humanos**: principal causa de falhas em data centers — complexidade **ultrapassa a capacidade cognitiva humana**

Manifesto IBM (2001)

"It's Time Systems Take Care of Themselves"

Admin define **o** quê —
sistema decide **como**

Inspiração: **Sistema Nervoso Autônomo (SNA)**

Projetos Precursores

Projeto	Ano / Origem	Contribuição principal
SAS	1997 — DARPA	Roteamento ad-hoc adaptativo em 10.000 nós
DASADA	2000 — DARPA	Probes e gauges para monitoramento; precursor direto do MAPE-K
IBM AC	2001 — IBM	Formalização do termo e das 4 propriedades Self-X
SPS	2003 — DARPA	Sistemas auto-regenerativos resistentes a ataques
ANTS	2005 — NASA	Enxame autônomo para exploração do espaço profundo

Projetos independentes que convergiram para os mesmos princípios de auto-gestão sem coordenação explícita

SAS — Sistema de Consciência Situacional (DARPA, 1997)

Objetivos

- ▶ Dispositivos pessoais de comunicação e localização para soldados no campo de batalha
- ▶ Disseminar relatórios de status autonomamente para até **10.000** soldados
- ▶ Latência abaixo de **200 ms** mesmo sob interferência inimiga
- ▶ Banda ampla: 20–2.500 MHz, 10 bps a 4 Mbps

Contribuição para AC

Primeiro sistema em larga escala de **roteamento ad-hoc adaptativo** multihop peer-to-peer; demonstrou **auto-configuração de topologia** dinâmica sob condições adversas

DASADA — Montagem Dinâmica para Adaptabilidade (DARPA, 2000)

Objetivos

- ▶ Pesquisar e desenvolver tecnologia para sistemas de missão crítica com alta **confiabilidade, dependabilidade e adaptabilidade**
- ▶ Tratar a complexidade de sistemas de software distribuídos de grande escala
- ▶ Objetivo similar ao da iniciativa de Computação Autônoma da IBM

Contribuição para AC

Pioneiro na abordagem **arquitetural** para auto-gestão; introduziu **probes e gauges** para monitoramento e motor de adaptação baseado em dados — **precursor direto do MAPE-K**

SPS — Sistemas Auto-Regenerativos (DARPA, 2003)

Objetivo: “desenvolver tecnologia para sistemas computacionais militares que forneçam funcionalidade crítica em qualquer momento, em face de danos por erros ou ataques”

4 aspectos técnicos:

1. Múltiplas versões funcionalmente equivalentes, implementadas diferentemente — ataque não afeta fração substancial do sistema
2. Modificações binárias dinâmicas (blocos aleatórios na pilha, endereços de branch variáveis)
3. Arquitetura de replicação tolerante a intrusões em larga escala
4. Estimativa de probabilidade de operador malicioso (*insider threat*)

Contribuição para AC

Formalizou **auto-proteção** e **auto-cura** em sistemas militares; base para sistemas tolerantes a ataques com **diversidade de implementação**

Propriedades Self-X

1. **Auto-configuração:** Sistema se instala e ajusta automaticamente segundo metas de alto nível. Um admin especifica o que deseja (ex: servidor web com tempo de resposta $< 2s$) e não como atingir. Adapta-se a plataformas e usuários sem passos manuais.

Propriedades Self-X

1. **Auto-configuração:** Sistema se instala e ajusta automaticamente segundo metas de alto nível. Um admin especifica o que deseja (ex: servidor web com tempo de resposta $< 2s$) e não como atingir. Adapta-se a plataformas e usuários sem passos manuais.
2. **Auto-cura:** detecta, diagnostica e corrige falhas de hardware e software; tolera falhas e reage aos seus primeiros sinais para garantir continuidade do serviço. Importante corrigir sem prejudicar o sistema.

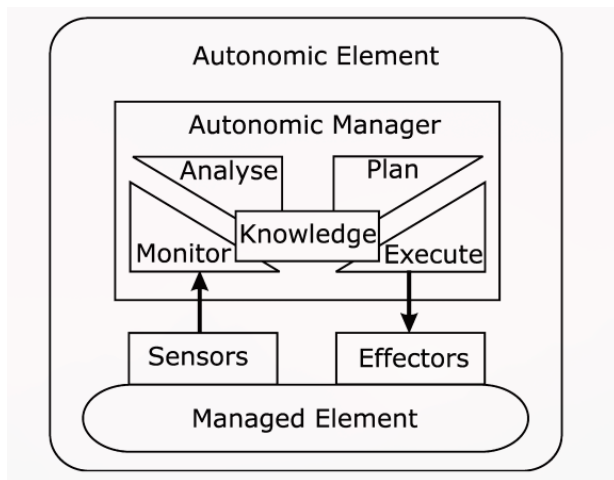
Propriedades Self-X (cont.)

3. **Auto-otimização:** busca contínua e *proativa* da eficiência máxima; Redistribui cargas para melhorar o desempenho ou então a *QoS*.

Propriedades Self-X (cont.)

3. **Auto-otimização:** busca contínua e *proativa* da eficiência máxima; Redistribui cargas para melhorar o desempenho ou então a *QoS*.
4. **Auto-proteção:** defende contra ataques maliciosos e mudanças prejudiciais; comportamento *proativo* (antecipa brechas) e *reativo* (responde a ameaças) para garantir integridade do sistema.

O Laço de Controle MAPE-K - Imagem



Componentes da Arquitetura MAPE-K

- ▶ **Managed Element:** recurso gerenciado (servidor, rede, aplicação). expõe interface de monitoramento e controle
- ▶ **Sensors:** coletam dados do Managed Element e os entregam ao Autonomic Manager
- ▶ **Effectors:** atuadores que aplicam as decisões do Autonomic Manager sobre o Managed Element.
- ▶ **Autonomic Manager:** componente central que executa o laço MAPE-K consumindo dados dos sensores e emitindo comandos pelos efetores.

Autonomic Manager

- ▶ Software configurável via metas de alto nível pelo administrador
- ▶ Usa dados dos sensores + conhecimento interno para planejar e executar as ações necessárias
- ▶ O conhecimento interno é tipicamente um modelo arquitetural do elemento gerenciado

Planejamento: Tipos de Políticas

- ▶ **ECA** (Evento-Condição-Ação) ("Se o evento ocorrer, e passar na codição, então faça isso"): diretas e simples; risco de conflito entre múltiplas regras detectável apenas em runtime como conflito por recurso.
- ▶ **Objetivo**: define estado desejado; sistema planeja *como* atingi-lo. Mais expressivo, exige maior raciocínio. Quando o estado desejado não é possível ser atingido, sistema não sabe como selecionar o melhor estado indesejado para ficar.
- ▶ **Função de Utilidade**: valor numérico por estado; otimiza mesmo quando o estado ideal é inatingível

Monitor: Sensores — Probes e Gauges

Probes

- ▶ Sensores específicos do sistema gerenciado
- ▶ Extraem dados brutos do *Managed Element*
- ▶ Especializados na implementação: Java/J2EE, C++, .NET

Gauges

- ▶ Filtram, agregam e processam dados dos probes
- ▶ Reportam ao Autonomic Manager para planejamento
- ▶ Especializados no **problema autônomo** (ex: otimizar disponibilidade de cluster web)

Correlação Causal

Gauges determinam **causa-raiz** correlacionando o instante em que diferentes probes dispararam e fazendo *pattern-matching* nos atributos — identificam sequência de causalidade da falha

Sensor Distribuído

Probes e gauges podem estar em **máquinas diferentes** do elemento gerenciado — requerem comunicação em rede entre si

Monitor: Monitoramento Passivo e Ativo

Passivo — sem instrumentação

- ▶ top, vmstat: CPU e memória por processo
- ▶ /proc: pseudo-FS com estado de runtime (Linux)
- ▶ Equivalentes no Windows 2000/XP

Ativo — com instrumentação

- ▶ Modificar/adicionar código ao SO ou à aplicação
- ▶ Capturar chamadas de função ou de sistema
- ▶ **ProbeMeister**: insere probes no bytecode Java compilado automaticamente

QMON (Agarwala et al., 2006)

Monitor **autônomo** que adapta sua própria frequência e volume de dados para *minimizar overhead* de monitoramento contínuo e *maximizar utilidade* dos dados coletados

Seleção de Métricas

Pequeno subconjunto de métricas fornece **90% de acurácia** na classificação do estado (Zhang & Figueiredo, 2006)

Analyse: *Stateless* vs *Stateful*

Análise *Stateless*

Decisão baseada **apenas** nos dados atuais dos sensores — sem memória de estado anterior

Limitação: gestor não reconhece padrões temporais nem tendências — reage, nunca antecipa

Análise *Stateful*

Autonomic Manager mantém estado do elemento gerenciado, atualizado progressivamente pelos sensores

Permite **raciocinar** sobre histórico, detectar anomalias e planejar com contexto completo

Abordagem *stateful* requer **modelo interno** do sistema — base para a abordagem por modelo arquitetural (*architectural model-driven approach*)

Analyse: Modelo Arquitetural

- ▶ Autonomic Manager cria e mantém modelo do elemento gerenciado: **comportamento, requisitos, metas e ambiente operacional**
- ▶ Modelo atualizado continuamente por dados dos sensores
- ▶ Adaptações planejadas no modelo **primeiro** — sistema real só é alterado se novo estado for válido
- ▶ Garante *integridade do sistema* após adaptação sem reinicialização

Componentes e Conectores

Componentes = tarefas computacionais concorrentes

Conectores = canais de comunicação

Sem restrição de granularidade: de um servidor Web completo a um único componente de aplicação

Problema: Defasagem

Delay entre mudança real e atualização do modelo → plano executado com base em estado desatualizado

Ciclo MAPE-K: Visão Geral

Ciclo completo

Monitor → **Analyse** → **Plan** → **Execute**, sustentado pela base de **Knowledge** compartilhada entre todas as fases

- ▶ Monitor coleta via probes/gauges
- ▶ Analyse raciocina sobre estado atual vs. metas — stateful com modelo arquitetural
- ▶ Plan define ações: ECA, objetivos ou função de utilidade
- ▶ Execute aplica via efetores (granularidade fina ou grossa)

Base de Conhecimento

- ▶ **Aprendizado por Reforço:** aprende políticas por tentativa e erro, vendo as consequências das ações; não exige modelo explícito do sistema gerenciado.
- ▶ **Redes Bayesianas:** decisão probabilística sob incerteza; seleção de algoritmos sensível a custo
- ▶ **Modelos Arquiteturais:** verifica validade das mudanças *antes* de executar no sistema real

Sumário

Histórico e Desenvolvimento

Sistemas Multitier e Graus de Autonomia

Aplicações

Estado da Arte

Conclusão

Graus de Autonomicidade: Modelo IBM

Nível	Nome	Descrição
1	Básico	Staff qualificado gerencia manualmente com ferramentas de monitoramento
2	Gerenciado	Ferramentas coletam informações de forma inteligente, reduzem carga administrativa
3	Preditivo	Monitoramento reconhece padrões de comportamento e <i>sugere</i> ações ao staff
4	Adaptativo	Sistema age com intervenção humana mínima; ajusta para cumprir os acordos em níveis de serviço
5	Autônomo	Componentes gerenciados dinamicamente por regras de negócio — plena autonomia

IBM estima que a maioria dos sistemas de TI atuais está nos níveis 1–2

Sumário

Histórico e Desenvolvimento

Sistemas Multitier e Graus de Autonomia

Aplicações

Estado da Arte

Conclusão

Cooperação entre Gerenciadores Autônomos

- ▶ Elementos autônomos podem **cooperar** para atingir objetivo comum (ex: minimizar tempo de resposta total do cluster)
- ▶ **Abordagem multi-agente:** cada gerenciador é um agente; comportamento global *emerge* da interação local
- ▶ **Abordagem hierárquica:** estruturação em níveis de abstração — alternativa à cooperação peer-to-peer
- ▶ Desafio central: garantir que objetivos locais de cada agente resultem no **objetivo global** sendo atingido

Objetivo da Cooperação

Design proposital de protocolos de interação para que resultado desejável e previsível *emerge* em nível superior sem coordenação central explícita

Emergência Engenheirada

- ▶ **Definição:** “design proposital de protocolos de interação de modo que resultado previsível e desejável é alcançado em nível superior” (Anthony, 2006)
- ▶ Inspiração: colônias de insetos — regras locais simples geram comportamento global complexo e robusto
- ▶ Cada nó opera com **informação local limitada**; nenhum nó conhece o estado global
- ▶ Contraste com algoritmo distribuído clássico: sem ACKs obrigatórios, sem ordenação rígida de eventos

Características

- ▶ Escala
- ▶ Robustez
- ▶ Estabilidade
- ▶ Sem ponto único de falha

Desafio

Garantir que o comportamento emergente seja o *desejado* — sistemas naturais não garantem isso

Aplicação Clássica: NASA ANTS — Contexto

- ▶ 1.000 nanoespaçonaves (<1 kg) no cinturão de asteróides
- ▶ 60–70% de perda esperada ao entrar no cinturão
- ▶ Atraso de comunicação com Terra inviabiliza controle humano em tempo real
- ▶ Papéis distribuídos: trabalhadores, coordenadores, mensageiros

Aplicação Clássica: NASA ANTS — Self-X

- ▶ **Auto-cura:** sobreviventes redistribuem papéis das naves perdidas automaticamente
- ▶ **Auto-config.:** cada nave determina posição na frota e rotas com vizinhos disponíveis
- ▶ **Auto-otimiz.:** prioriza asteróides com base nos dados já coletados
- ▶ **Auto-proteção:** evita colisões; gerencia consumo de energia das velas solares

MAPE-K distribuído: cada nave executa ciclo local;
comportamento global coerente emerge do enxame

Aplicação Clássica: IBM *Project eLiza*

- ▶ Maior foco individual de investimento da divisão de servidores IBM — linha *eServer* (zSeries, pSeries, iSeries, xSeries)
- ▶ Espinha dorsal de operações bancárias, de telecomunicações e governamentais globais
- ▶ **Auto-config.:** hardware e middleware se reconfiguram na inicialização e em runtime sem intervenção do operador
- ▶ **Auto-otimização:** *Intelligent Resource Director* (IRD) redistribui cargas entre partições lógicas (LPARs) para maximizar throughput e cumprir SLAs

Aplicação Clássica: IBM *Project eLiza* (cont.)

- ▶ **Auto-cura:** *System Automation for OS/390* detecta falhas em aplicações e aciona recuperação automática — inicia, para, monitora e recupera serviços sem humano
- ▶ **Auto-proteção:** detecção de intrusão e isolamento de falhas integrados diretamente ao SO

Provou que Self-X é viável em sistemas de missão crítica em produção real

Orquestração e Auto-scaling em Nuvem

A Computação Autônoma atua como o motor de orquestração essencial para manter a viabilidade econômica dos provedores modernos.

- ▶ **Auto-otimização:** Provisionamento dinâmico (auto-scaling) baseado em métricas de CPU e latência em tempo real. Economizando energia e custo.
- ▶ **Eficiência Energética:** Consolidação de cargas de trabalho e desligamento de recursos ociosos.
- ▶ **Resiliência:** Detecção de degradação e migração automática de VMs sem interrupção de serviço. Por meio de migração automática de VMs e reconfiguração de balanceadores.

Orquestração e Auto-scaling em Nuvem

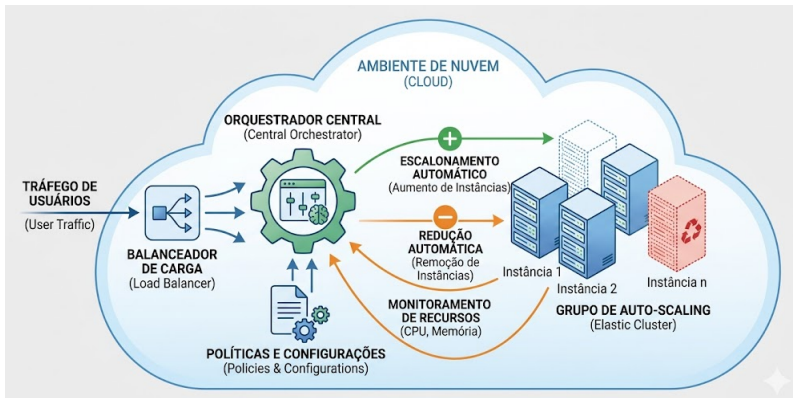


Figura: Funcionamento da orquestração em nuvem

Sistemas Pervasivos e IoT

- ▶ Desafio: heterogeneidade + volatilidade + recursos limitados.
- ▶ Auto-configuração: dispositivos plug-and-play em Smart Cities e Smart Grids. Descobrem vizinhos e registram serviços automaticamente.
- ▶ Auto-proteção: detecção de tráfego anômalo (DDoS, intrusões) e quarentena automática de dispositivos comprometidos.
- ▶ Sem autonomia, o custo de configurar milhões de sensores seria proibitivo.

Sumário

Histórico e Desenvolvimento

Sistemas Multitier e Graus de Autonomia

Aplicações

Estado da Arte

Conclusão

Estado da Arte: Mudança de Paradigma

A pesquisa não busca mais justificar a autonomia, mas refinar o MAPE-K e resolver suas limitações.

Principal mudança no método foi sair de regras estáticas para a autonomia preditiva com IA/ML.

- ▶ Aprendizado por Reforço e Redes Neurais Profundas substituem políticas estáticas se-então.
- ▶ O sistema aprende políticas ótimas por tentativa e erro ou via dados históricos.

Paradigmas Industriais Consolidados

AIOps (Artificial Intelligence for IT Operations)

- ▶ Estado da arte em auto-cura e auto-otimização de data centers.
- ▶ Ingere terabytes de logs/métricas e antecipa falhas horas antes de ocorrerem.

Orquestração Cloud-Native & Service Mesh

- ▶ Microserviços adotam Self-X de forma nativa.
- ▶ Kubernetes + Service Mesh → auto-configuração e auto-cura em milissegundos.

Lacunas e Desafios em Aberto

1. Verificação Formal e Estabilidade

Problema da “caixa-preta” do ML → necessidade de provar matematicamente a corretude.

2. Interoperabilidade & Sistemas Multi-agente

Soluções atuais vivem em silos; falta padronização para troca de conhecimento entre gerentes autônomos.

3. Tradução de Políticas de Alto Nível

Converter metas de negócio (“maximizar lucro minimizando latência”) em ações operacionais ainda exige especialista humano.

Fronteira atual: tornar a autonomia verificável, interpretável e interoperável.

Sumário

Histórico e Desenvolvimento

Sistemas Multitier e Graus de Autonomia

Aplicações

Estado da Arte

Conclusão

Conclusão

- ▶ A escalabilidade da TI moderna não pode mais ser dissociada da automação de sua gestão.
- ▶ A capacidade cognitiva humana é o principal limitador das infraestruturas contemporâneas.
- ▶ A “crise da complexidade” é o desafio central. A Computação Autônoma é a resposta.
- ▶ As propriedades Self-X, sustentadas pelo MAPE-K, viabilizam Cloud, IoT e sistemas pervasivos não apenas tecnicamente, mas economicamente (redução de TCO + cumprimento de SLAs).

Conclusão

A transição de sistemas que auxiliam decisões → para sistemas que agem sozinhos ainda esbarra em:

- ▶ Interoperabilidade entre fornecedores
- ▶ Confiança dos administradores
- ▶ Estabilidade formal dos algoritmos

Em uma era de trilhões de dispositivos, autonomia deixa de ser diferencial e passa a ser requisito mandatório.

Referências Principais

- ▶ IBM (2001). *Autonomic Computing: An Architectural Blueprint*
- ▶ Kephart & Chess (2003). *The Vision of Autonomic Computing*. IEEE Computer
- ▶ Sterritt (2005). *Autonomic Computing*. Innovations in Systems and Software Engineering
- ▶ Lalanda et al. (2013). *Autonomic Computing: Principles, Design and Implementation*
- ▶ Ganek & Corbi (2003). *The dawning of the autonomic computing era*. IBM Systems Journal
- ▶ Truszkowski et al. (2004). *Autonomous and Autonomic Systems*

Obrigado!

P&R 1: O que motivou a Computação Autônoma?

Pergunta

Qual foi a principal motivação para o surgimento da Computação Autônoma e qual foi o papel da IBM?

Resposta

A “crise da complexidade”: TCO com até 80% em manutenção e erros humanos como principal causa de falhas em data centers. Em 2001, a IBM publicou manifesto propondo sistemas auto-gerenciáveis inspirados no Sistema Nervoso Autônomo (SNA) — administrador define metas de alto nível, sistema decide como atingi-las.

P&R 2: Quais são as propriedades Self-X?

Pergunta

Quais são as 4 propriedades Self-X e como elas se complementam?

Resposta

- ▶ **Auto-config.:** sistema se instala e ajusta sem passos manuais
- ▶ **Auto-cura:** detecta, diagnostica e corrige falhas de HW/SW
- ▶ **Auto-otimização:** redistribui recursos *proativamente* para cumprir SLAs
- ▶ **Auto-proteção:** defende contra ataques — proativa e reativa

P&R 3: Como funciona o ciclo MAPE-K?

Pergunta

Descreva o ciclo MAPE-K e o papel da base de Conhecimento.

Resposta

Monitor coleta → **Analyse** identifica desvios vs. metas → **Plan** define ações → **Execute** aplica via efetores.

Knowledge sustenta todas as fases: políticas, histórico e modelos arquiteturais
— valida mudanças antes de executar no sistema real.

P&R 4: Computação Autônoma em Cloud e IoT

Pergunta

Como a Computação Autônoma se manifesta em Cloud Computing e IoT?

Resposta

Cloud: auto-scaling por MAPE-K; falha de nó → migração automática de VMs.

IoT: sensores com *plug-and-play*; dispositivos comprometidos isolados em quarentena automaticamente.

P&R 5: Desafios para Adoção Plena

Pergunta

Quais são os principais desafios que impedem a adoção plena da Computação Autônoma?

Resposta

1. **Verificação Formal:** ML é “caixa-preta” — difícil provar correteude matematicamente
2. **Interoperabilidade:** soluções em silos; falta protocolo para troca de Knowledge entre gerentes
3. **Tradução de Políticas:** metas de negócio ainda exigem especialista para mapear em config. operacional